

VELORA GPU 학습 운영 및 반출 가이드

폐쇄망 GPU 서버 운영 기준

1. 목적

이 문서는 폐쇄망 GPU 서버에서 VELORA 인지기능 음성 모델을 학습하고, 최종 결과물을 반출하기 위한 사용법을 정리한 문서입니다.

모델은 AI Hub 원천 데이터의 `.flac` 음성 파일과 `.json` 어노테이션을 사용하여 `Normal / MCI / AD` 3분류 모델을 학습합니다. 서비스에서는 이 결과를 의료 진단이 아니라 비의료적 인지기능 변화 위험 신호 참고 지표로 사용합니다.

2. 기본 폴더 구조

GPU 서버의 기본 구조는 아래와 같습니다.

```
~/workspace/  
dataset/  
  IB-APPS/  
  CERAD-K/  
  SNSB-II/  
nasw/  
  07-prepare_normal_mci_ad_dataset.py  
  08-train_normal_mci_ad_vgg16.py  
  09-infer_normal_mci_ad.py  
  10-train_from_raw_dataset.py  
  11-create_sample_dataset.py  
  environment.yml  
  requirements-server.txt  
download/
```

`dataset`에는 원천 raw 데이터가 들어갑니다. `nasw`에는 학습 코드가 들어갑니다. `download`에는 변환 데이터, 학습 모델, 학습 로그가 생성됩니다.

3. 환경 활성화 및 점검

```
cd ~/workspace/nasw  
conda activate velora-cognitive-voice
```

폐쇄망 이전에 환경을 미리 준비하려면 `environment.yml`을 사용하여 로컬에서 conda 환경을 만들고, `conda-pack`으로 아카이브해 두는 것이 안전합니다.

```
cd ~/workspace/nasw  
conda env create -f environment.yml  
conda activate velora-cognitive-voice  
conda install -c conda-forge conda-pack  
conda pack -o download/velora-cognitive-voice.tar.gz
```

생성된 `download/velora-cognitive-voice.tar.gz`는 폐쇄망 서버로 복사하여 사용할 수 있습니다.

GPU 인식 확인:

```
nvidia-smi  
python -c "import tensorflow as tf; print(tf.__version__); print(tf.config."
```

```
list_physical_devices('GPU'))"
```

정상 예:

```
[PhysicalDevice(name='/physical_device:GPU:0', device_type='GPU')]
```

오디오 처리 패키지 확인:

```
ffmpeg -version  
python -c "import librosa, soundfile, matplotlib, pandas, PIL; print('packages ok')"
```

4. 실제 데이터 사전 점검

AI Hub 실제 데이터가 들어오면 먼저 구조와 대표 파일을 확인합니다.

```
ls -la ~/workspace/dataset  
find ~/workspace/dataset -maxdepth 2 -type d | head -40  
find ~/workspace/dataset -name '*_R.flac' | head  
find ~/workspace/dataset -name '*_R.json' | head
```

기대 폴더:

```
IB-APPS  
CERAD-K  
SNSB-II
```

기본 학습 음성은 `\_R.flac` 또는 `raw-file`입니다. 라벨은 `\_R.json` 안의 `subject\_type`을 사용합니다.

```
subject_type = 1 -> Normal  
subject_type = 5 -> MCI  
subject_type = 10 -> AD
```

데이터셋 구조가 위와 다를 수도 있습니다. 예를 들어 다음처럼 한 단계 더 깊게 압축이 풀리거나, 원천 데이터와 라벨링 데이터가 분리되어 있을 수 있습니다.

```
~/workspace/dataset/압축해제폴더/Training/...  
~/workspace/dataset/원천데이터/...  
~/workspace/dataset/라벨링데이터/...
```

이 경우에도 `07-prepare\_normal\_mci\_ad\_dataset.py`는 `--source-root` 아래를 재귀적으로 탐색합니다. 먼저 `IB-APPS`, `CERAD-K`, `SNSB-II` 폴더명을 찾고, 찾지 못하면 전체 JSON 파일을 탐색합니다. JSON과 음성 파일이 서로 다른 하위 폴더에 있으면 `.flac`, `.wav`, `.mp3`, `.m4a` 파일명을 색인하여 JSON에 적힌 파일명과 다시 매칭합니다.

가장 안전한 방법은 라벨 JSON과 음성 파일을 모두 포함하는 가장 상위 폴더를 `--source-root`로 지정하는 것입니다.

```
python 07-prepare_normal_mci_ad_dataset.py \  
  --source-root ~/workspace/dataset \  
  --manifest-only
```

다른 위치에 데이터셋이 있으면 다음처럼 직접 지정합니다.

```
python 07-prepare_normal_mci_ad_dataset.py \  
  --source-root /path/to/delivered_dataset_root \  
  --manifest-only
```

실행 출력의 `source\_roots\_scanned`, `json\_roots\_scanned`를 먼저 확인합니다. `read` 또는 `written`이 0이면 실제 JSON과 음성 파일이 이 탐색 범위 안에 있는지 확인해야 합니다.

데이터가 매우 커서 음성 파일명 색인이 부담되면 다음 옵션으로 비활성화할 수 있습니다.

```
python 07-prepare_normal_mci_ad_dataset.py \  
  --source-root ~/workspace/dataset \  
  --manifest-only \  
  --no-recursive-audio-search
```

5. Manifest 점검

이미지 생성 전에 JSON, 라벨, 음성 경로 매칭만 먼저 확인합니다.

```
python 07-prepare_normal_mci_ad_dataset.py --manifest-only
```

출력에서 다음 값을 확인합니다.

```
read: 0보다 커야 함  
written: 0보다 커야 함  
skipped_unknown_label: 높으면 라벨 누락 확인  
skipped_missing_audio: 높으면 flac 경로 확인  
json_errors: 0 또는 원인 확인
```

Manifest 위치:

```
~/workspace/nasw/download/prepared_normal_mci_ad/dataset_manifest.csv
```

전처리 설정 파일 위치:

```
~/workspace/nasw/download/prepared_normal_mci_ad/preprocessing_config.json
```

이 파일에는 `sample\_rate`, `seconds`, `dpi`, `audio\_kind`, split 비율이 저장됩니다. 학습 metadata에도 포함되며, 추론 시 자동으로 재사용됩니다.

6. Mel Spectrogram 데이터셋 생성

Manifest 점검이 정상이면 `.flac` 음성을 Mel spectrogram `.jpg` 이미지로 변환합니다.

```
python 07-prepare_normal_mci_ad_dataset.py --overwrite
```

생성 위치:

```
~/workspace/nasw/download/prepared_normal_mci_ad/
```

학습용 class 폴더 확인:

```
find ~/workspace/nasw/download/prepared_normal_mci_ad/multiclass/ALL -maxdepth 2 -type d | sort
```

기대 구조:

```
train/AD  
train/MCI  
train/Normal  
validation/AD  
validation/MCI  
validation/Normal  
test/AD  
test/MCI
```

test/Normal

split 비율 조건:

```
0 < train_ratio < 1  
0 <= val_ratio < 1  
train_ratio + val_ratio < 1
```

7. 실제 학습 실행

인터넷이 열려 있고 ImageNet 가중치를 사용할 수 있으면 다음을 권장합니다.

```
python 10-train_from_raw_dataset.py --overwrite --tasks ALL --weights imagenet
```

폐쇄망이거나 ImageNet 가중치가 없으면 다음을 사용합니다.

```
python 10-train_from_raw_dataset.py --overwrite --tasks ALL --weights none
```

`imagenet`은 VGG16을 ImageNet 사전학습 가중치로 초기화한다는 뜻입니다. `none`은 랜덤 초기화로 학습한다는 뜻입니다.

8. 샘플 데이터로 사전 테스트

실제 데이터가 아직 없을 때는 샘플 데이터를 만들어 환경과 파이프라인을 점검할 수 있습니다.

```
cd ~/workspace/nasw
python 11-create_sample_dataset.py --output-root ~/workspace/nasw/download/sample_dataset --overwrite
python 10-train_from_raw_dataset.py \
  --source-root ~/workspace/nasw/download/sample_dataset \
  --prepared-root ~/workspace/nasw/download/prepared_normal_mci_ad \
  --output-root ~/workspace/nasw/download/ad_mci_normal \
  --overwrite \
  --tasks ALL \
  --weights none \
  --epochs 1 \
  --batch-size 4
```

샘플 데이터는 환경 점검용이며 모델 정확도 판단에 사용하면 안 됩니다.

9. 장시간 학습 재개

데이터가 많아 시간이 오래 걸리면 데이터 준비와 학습을 분리합니다.

데이터 준비:

```
python 07-prepare_normal_mci_ad_dataset.py --overwrite
```

학습 시작:

```
python 08-train_normal_mci_ad_vgg16.py --tasks ALL --weights imagenet
```

학습 중 저장되는 파일:

```
download/ad_mci_normal/task-ALL/normal_mci_ad_task-ALL_best.h5
download/ad_mci_normal/task-ALL/normal_mci_ad_task-ALL_final.h5
```

중단 후 최신 checkpoint에서 재개:

```
python 08-train_normal_mci_ad_vgg16.py --tasks ALL --resume --resume-from final
```

best checkpoint에서 재개:

```
python 08-train_normal_mci_ad_vgg16.py --tasks ALL --resume --resume-from best
```

Spectrogram 재생성을 건너뛰고 재개:

```
python 10-train_from_raw_dataset.py --skip-prepare --tasks ALL --resume --resume-from final
```

특정 모델 파일에서 시작:

```
python 08-train_normal_mci_ad_vgg16.py \  
  --tasks ALL \  
  --initial-model ~/workspace/nasw/download/ad_mci_normal/task-ALL/normal_mci_ad_task-ALL_best.  
h5
```

10. 학습 결과 확인

결과 위치:

```
~/workspace/nasw/download/ad_mci_normal/task-ALL/
```

확인:

```
ls -lh ~/workspace/nasw/download/ad_mci_normal/task-ALL/  
cat ~/workspace/nasw/download/ad_mci_normal/training_summary.json
```

기대 파일:

```
normal_mci_ad_task-ALL_best.h5  
normal_mci_ad_task-ALL_final.h5  
normal_mci_ad_task-ALL_metadata.json  
normal_mci_ad_task-ALL_<timestamp>.csv
```

11. 추론 테스트

```
AUDIO=$(find ~/workspace/dataset -name '*_R.flac' | head -1)  
python 09-infer_normal_mci_ad.py --audio-file "$AUDIO"
```

`--model`을 생략하면 `download/ad\_mci\_normal` 아래의 최신 best 모델을 자동으로 찾습니다.

12. 백엔드 연결 환경변수

```
export VELORA_COGNITIVE_MODEL_PATH=~/.workspace/nasw/download/ad_mci_normal/task-ALL/normal_mci_ad_task-ALL_best.h5
export VELORA_COGNITIVE_METADATA_PATH=~/.workspace/nasw/download/ad_mci_normal/task-ALL/normal_mci_ad_task-ALL_metadata.json
```

첫 번째 변수는 추론에 사용할 모델 `.h5` 경로입니다. 두 번째 변수는 class 순서, image size, 전처리 설정이 담긴 metadata 경로입니다.

13. 최종 반출 필수 파일

반드시 반출해야 하는 파일은 2개입니다.

```
normal_mci_ad_task-ALL_best.h5
normal_mci_ad_task-ALL_metadata.json
```

위치:

```
~/.workspace/nasw/download/ad_mci_normal/task-ALL/
```

반출용 폴더 생성:

```
mkdir -p ~/velora_export
cp ~/.workspace/nasw/download/ad_mci_normal/task-ALL/normal_mci_ad_task-ALL_best.h5 ~/velora_export/
cp ~/.workspace/nasw/download/ad_mci_normal/task-ALL/normal_mci_ad_task-ALL_metadata.json ~/velora_export/
```

선택 반출 파일:

```
training_summary.json
normal_mci_ad_task-ALL_<timestamp>.csv
```

선택 파일 복사:

```
cp ~/.workspace/nasw/download/ad_mci_normal/training_summary.json ~/velora_export/
cp ~/.workspace/nasw/download/ad_mci_normal/task-ALL/*.csv ~/velora_export/
```

압축:

```
cd ~
tar -czf velora_model_export.tar.gz velora_export
```

최종 반출 파일:

```
~/velora_model_export.tar.gz
```

14. 반출 비권장 대상

아래 항목은 원천 데이터 또는 원천 데이터의 파생물일 수 있으므로 반출하지 않는 것을 권장합니다.

```
~/workspace/dataset/
~/workspace/nasw/download/prepared_normal_mci_ad/
~/workspace/nasw/download/sample_dataset/
```

`download` 전체 반환은 권장하지 않습니다. 실제 서비스 연결에는 `best.h5`와 `metadata.json`이 핵심입니다.

15. GitHub 반영

현재 서버에서 GitHub로 업로드:

```
cd /root/velora_train
git add README.md 07-prepare_normal_mci_ad_dataset.py 08-train_normal_mci_ad_vgg16.py 09-
infer_normal_mci_ad.py 10-train_from_raw_dataset.py 11-create_sample_dataset.py
git commit -m "Update GPU training pipeline"
GIT_SSH_COMMAND='ssh -i /root/velora_train/.ssh/id_ed25519 -o UserKnownHostsFile=/tmp/
github_known_hosts -o StrictHostKeyChecking=accept-new' git push
```

GPU 서버에서 최신 코드 받기:

```
cd ~/workspace/nasw
git pull origin main
```